SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA1409 - COMPILER DESIGN

LEX Programs 11–20

Name: E.Ragul

Register Number: 192571032

Department: B.Tech Computer Science and BioScience

SIMATS Engineering, Chennai

Experiment No. 11

Experiment Name: Implement Lexical Analyzer using FLEX

Aim

To write a LEX program to identify and display tokens in a given C source program.

Procedure

1. Create a C source file named sample.c.
2. Write the LEX program and save it as token.l.
3. Compile using flex token.l.
4. Compile the generated file using gcc lex.yy.c.
5. Execute using a.exe sample.c.
6. Display the identified tokens with appropriate captions.

Program: token.l

```
%{
#include <stdio.h>
int count_id = 0, count_key = 0;
%}

digit [0-9]
letter [A-Za-z]

%%
(stdio\.h|conio\.h) {
   printf("%s is a standard library\n", yytext);
}

(include|void|main|printf|int) {
   printf("%s is a keyword\n", yytext);
   count_key++;
}
```

```
{letter}({letter}|{digit})* {
   printf("%s is an identifier\n", yytext);
   count_id++;
}
{digit}+ {
   printf("%s is a number\n", yytext);
}
\"([^\"\\\n]|\\.)*\" {
   printf("%s is a string literal\n", yytext);
}
[ \t\n#<>(){};,=+*/&]+  { }
%%
int yywrap(void)
int main(int argc, char *argv[])
{
   if (argc < 2)
   {
      printf("Usage: a.exe sample.c\n");
      return 1;
   }
   yyin = fopen(argv[1], "r");
   if (yyin == NULL)
   yylex();
   printf("Number of identifiers = %d\n", count_id);
   printf("Number of keywords = %d\n", count_key);
   fclose(yyin);
   return 0;
}
```

Input Source Program: sample.c

```
#include <stdio.h>

void main()

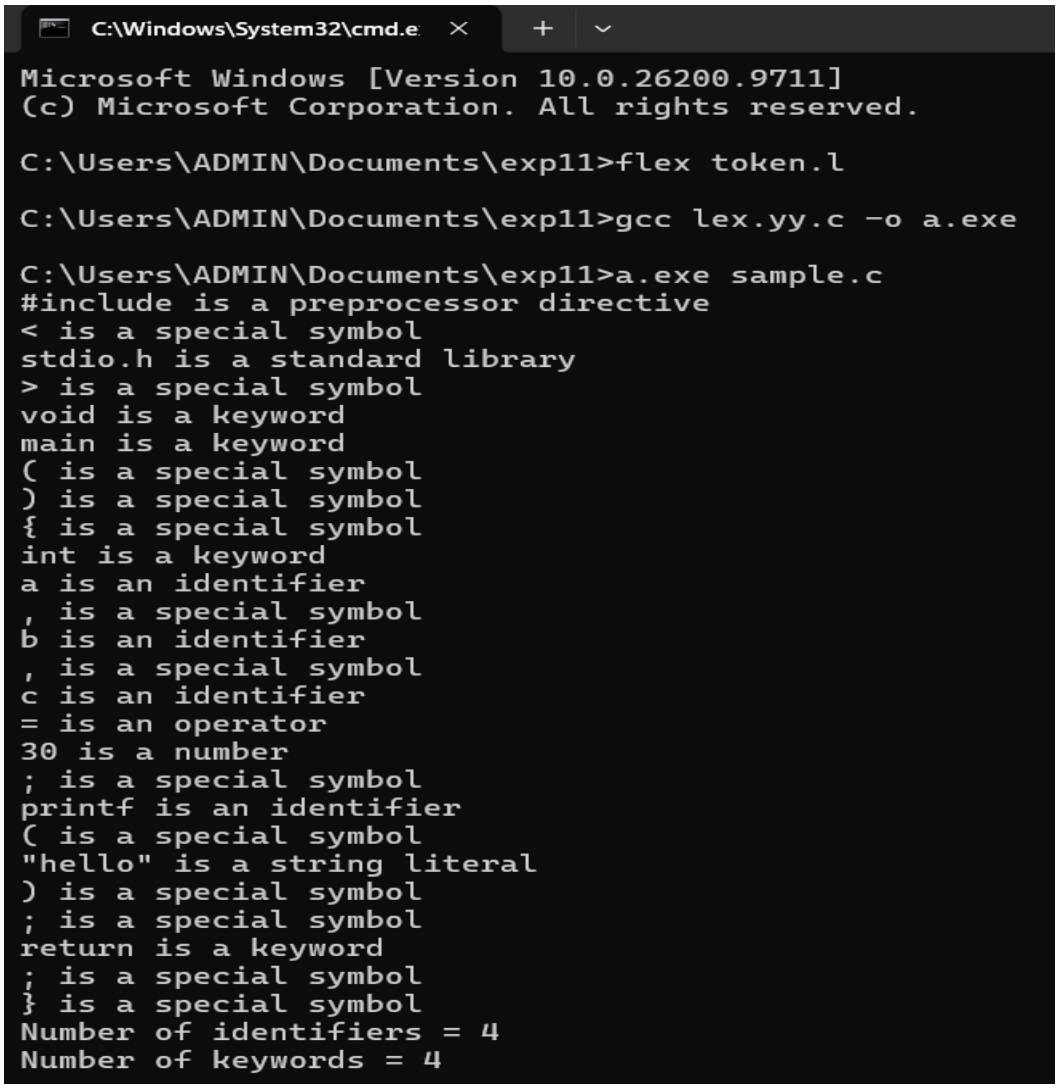
{

   int a, b, c = 30;

   printf("hello");

}
```

Output



Result

Thus, the LEX program was written and executed to identify and display tokens in a given C source program using FLEX.

Experiment No. 12

Experiment Name: Count Vowels and Consonants using LEX

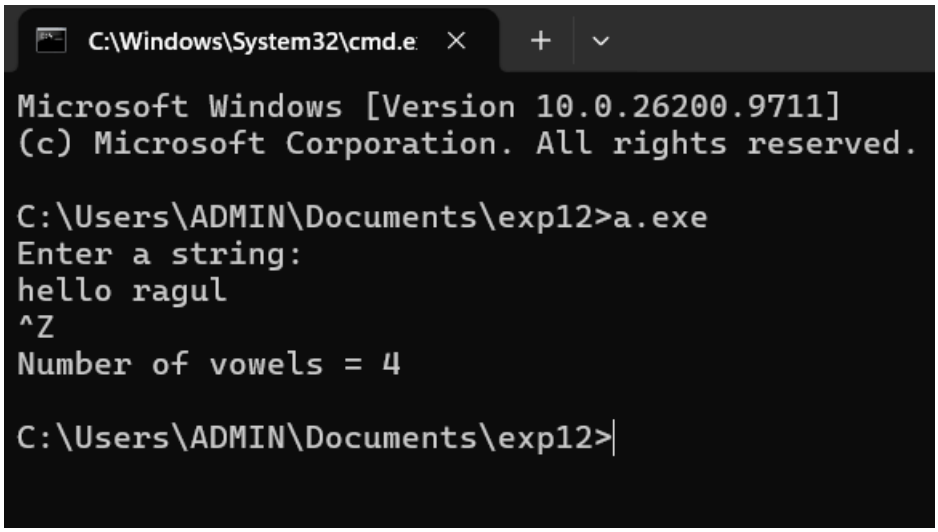
Aim:

To write a LEX program to count vowels and consonants in a given string.

Program: (vowels.l)

```
%{
#include <stdio.h>
int vowels = 0, consonants = 0;
%}
%%
[aeiouAEIOU]  { vowels++; }
[a-zA-Z]      { consonants++; }
.|\n          { }
%%
int yywrap()
{
   return 1;
}
int main()
{
   printf("Enter a string: ");
   yylex();
   printf("Number of vowels = %d\n", vowels);
   printf("Number of consonants = %d\n", consonants);
   return 0;
}
```

Output:



Result:

Thus, the LEX program to count vowels and consonants was executed successfully.

Experiment No. 13

Experiment Name: Count Vowels and Consonants using LEX

Aim:

To write a LEX program to count vowels and consonants in a given input string.

Program: (vowels.l)

```
%{
#include <stdio.h>
int v = 0, c = 0;
%}

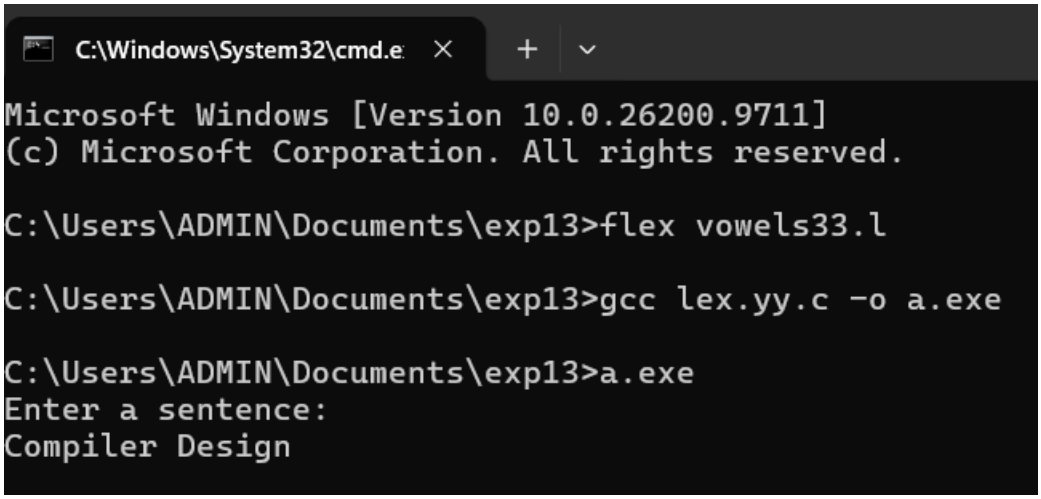
%%
[aeiouAEIOU]  { v++; }
[a-zA-Z]      { c++; }
.|\n          { }
%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Enter a string: ");
   yylex();
   printf("Vowels = %d\n", v);
   printf("Consonants = %d\n", c);
   return 0;
```

}

Output:



Result:

Thus, the LEX program to count vowels and consonants was executed successfully.

Experiment No. 14

Experiment Name: Identify Keywords and Identifiers using LEX

Aim:

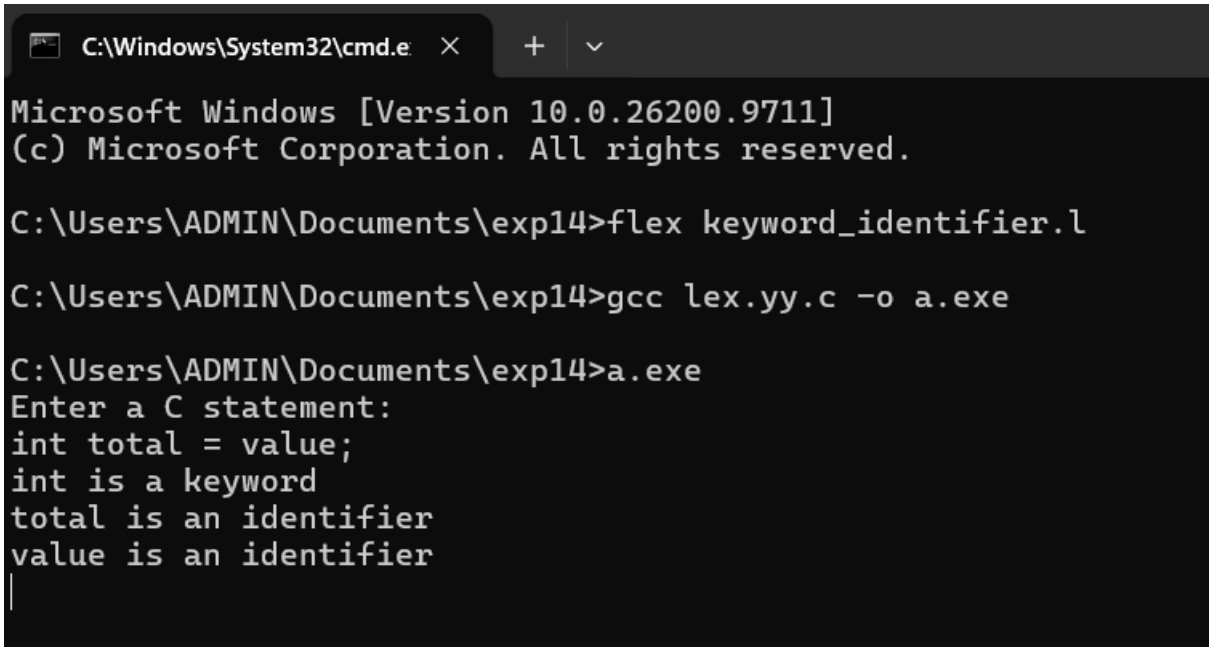
To write a LEX program to identify keywords and identifiers in a given input.

Program: (keyword_identifier.l)

```
%{
#include <stdio.h>
#include <string.h>
int keywords = 0, identifiers = 0;
char *words[] = {
    "int", "float", "char", "if", "else",
    "while", "for", "return", "void", "main"
};
int isKeyword(char *s)
{
    int i;
    for (i = 0; i < 10; i++)
        if (strcmp(s, words[i]) == 0)
            return 1;
    return 0;
%%
[a-zA-Z_][a-zA-Z0-9_]* {
    if (isKeyword(yytext)) {
        printf("%s is a keyword\n", yytext);
        keywords++;
    } else {
        printf("%s is an identifier\n", yytext);
        identifiers++;
    }
```

```
}
[ \t\n]+  ;
.        ;
%%
int yywrap()
int main()
{
   printf("Enter a statement: ");
   yylex();
   printf("Number of keywords = %d\n", keywords);
   printf("Number of identifiers = %d\n", identifiers);
   return 0;
}
```

Output:



Result:

Thus, the LEX program to identify keywords and identifiers was executed successfully.

Experiment No. 15

Experiment Name: Recognize Numbers and Words using LEX

Aim:

To write a LEX program to recognize numbers and words in a given input.

Program: (numbers_words.l)

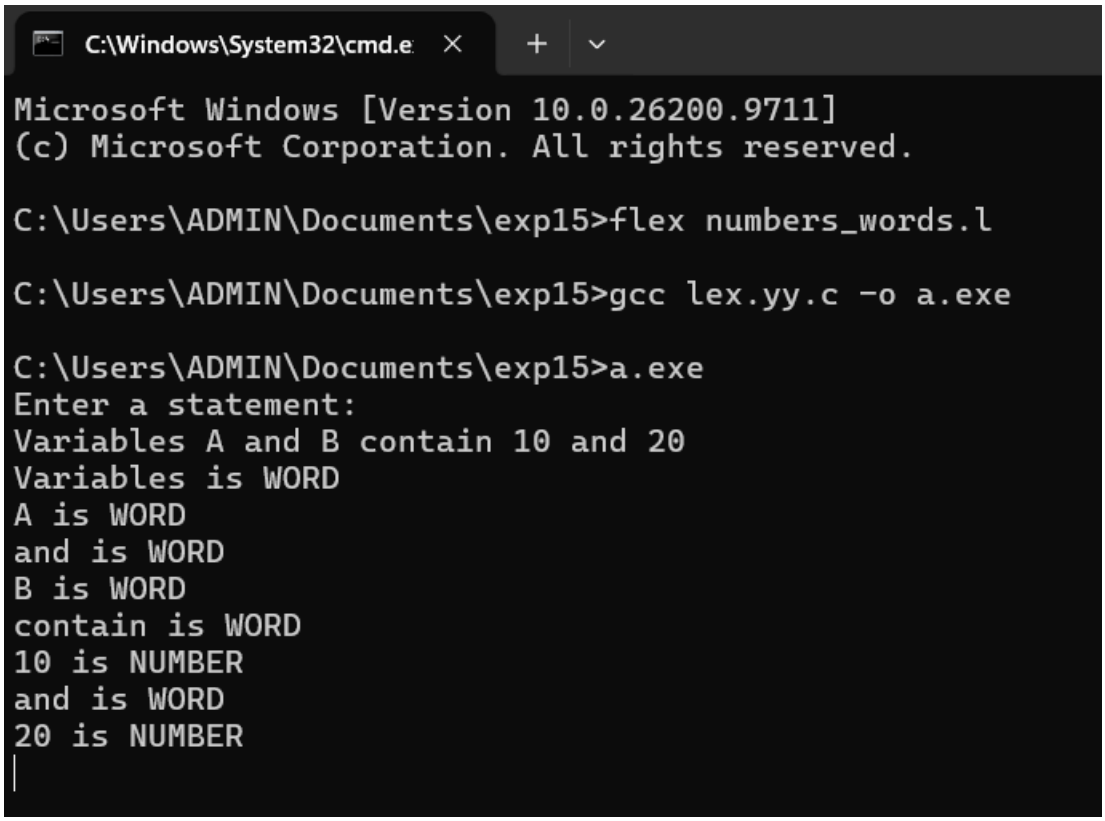
```
%{
#include <stdio.h>
%}

%%
[0-9]+        { printf("%s is a number\n", yytext); }
[a-zA-Z]+     { printf("%s is a word\n", yytext); }
[ \t\n]+      ;
.             ;
%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Enter a string: ");
   yylex();
   return 0;
}
```

Output:



Result:

Thus, the LEX program to recognize numbers and words was executed successfully.

Experiment No. 16

Experiment Name: Identify Positive and Negative Numbers using LEX

Aim:

To write a LEX program to identify positive and negative numbers.

Program: (positive_neg_nums.l)

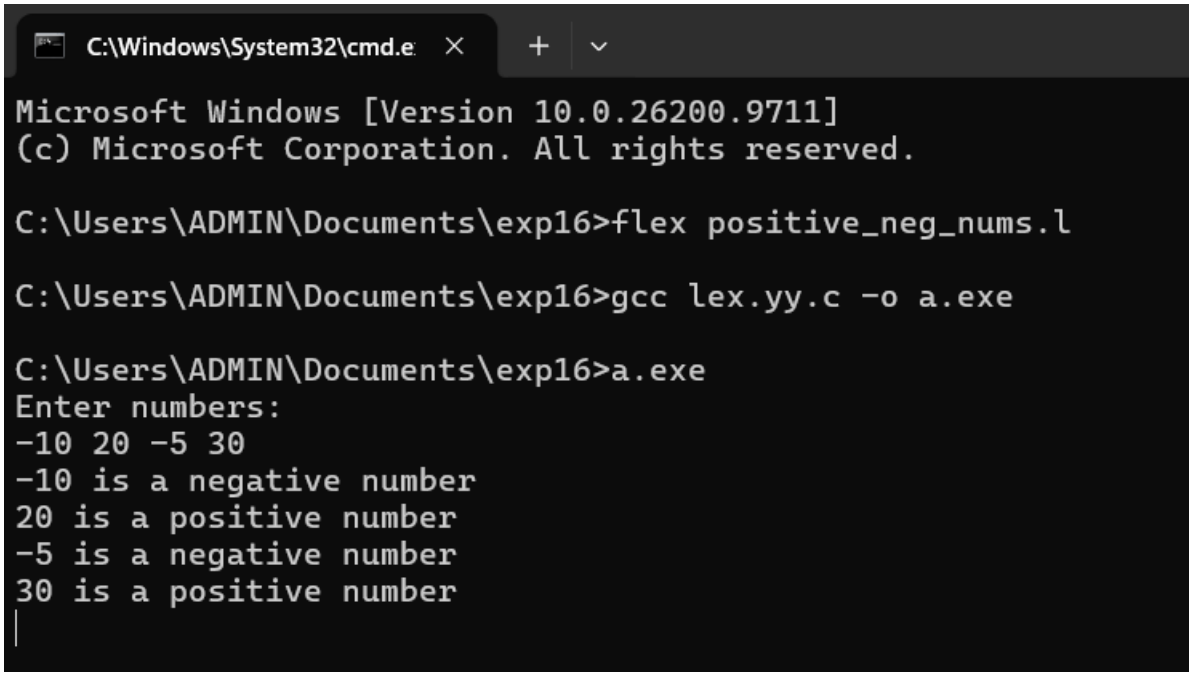
```
%{
#include <stdio.h>
int positive = 0, negative = 0;
%}

%%
-[0-9]+ {
   printf("%s is a negative number\n", yytext);
   negative++;
}
[0-9]+ {
   printf("%s is a positive number\n", yytext);
   positive++;
}
[ \t\n]+ ;
.       ;
%%

int yywrap()
{
   return 1;
}
```

```
int main()
{
   printf("Enter numbers: ");
   yylex();
   printf("Positive numbers = %d\n", positive);
   printf("Negative numbers = %d\n", negative);
   return 0;
}
```

Output:



Result:

Thus, the LEX program to identify positive and negative numbers was executed successfully.

Experiment No. 17

Experiment Name: Validate URL using LEX

Aim:

To write a LEX program to check whether a given URL is valid.

Program: (url.l)

```
%{
#include <stdio.h>
%}

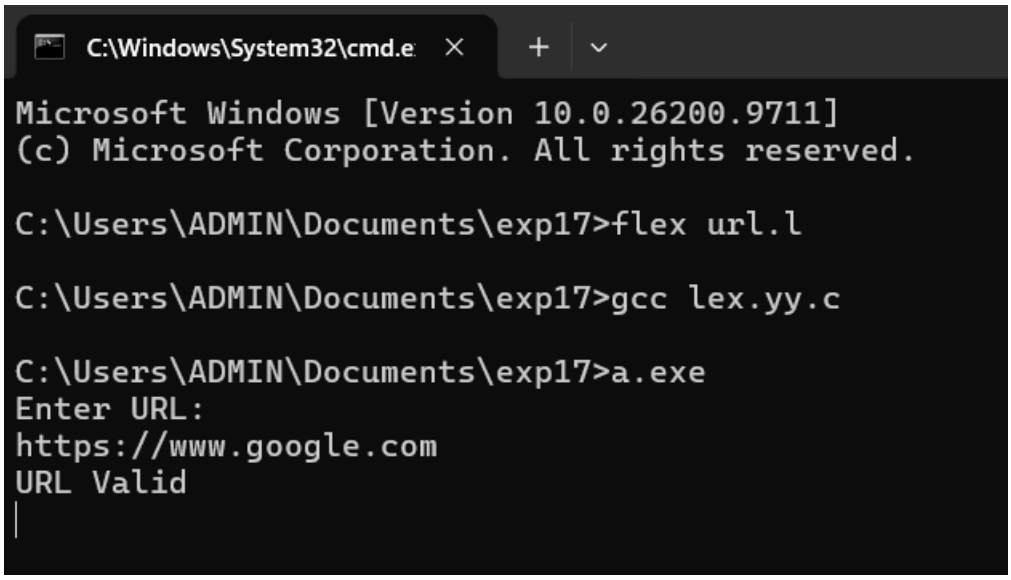
%%
https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(/[^ \t\n]*)? {
    printf("Valid URL\n");
}
[^ \t\n]+ {
    printf("Invalid URL\n");
}
[ \t\n]+ ;
%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter a URL: ");
    yylex();
```

```
    return 0;
}
```

Output:



Result:

Thus, the LEX program to validate a URL was executed successfully.

Experiment No. 18

Experiment Name: Validate Date of Birth using LEX

Aim:

To write a LEX program to check whether a date of birth follows the format DD/MM/YYYY.

Program: (dob.l)

```
%{
#include <stdio.h>
%}

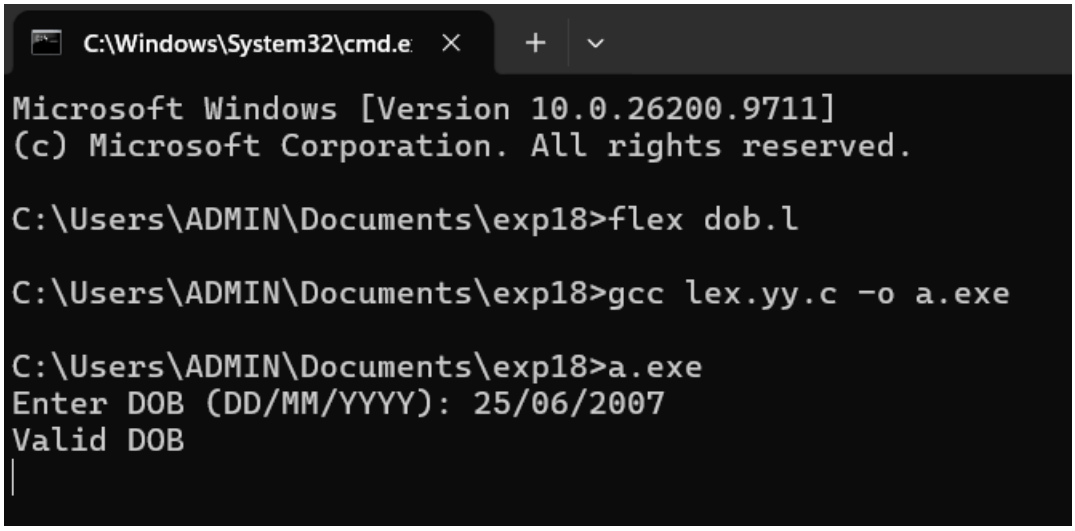
%%
[0-9]{2}/[0-9]{2}/[0-9]{4} {
   printf("Valid DOB format\n");
}
[^\n]+ {
   printf("Invalid DOB format\n");
}
%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Enter DOB (DD/MM/YYYY): ");
   yylex();
   return 0;
```

}

Output:



Result:

Thus, the LEX program to validate the date of birth format was executed successfully.

Experiment No. 19

Experiment Name: Check Whether Input is a Digit using LEX

Aim:

To write a LEX program to check whether the given input is a digit.

Program: (digit_or_not.l)

```
%{
#include <stdio.h>
%}

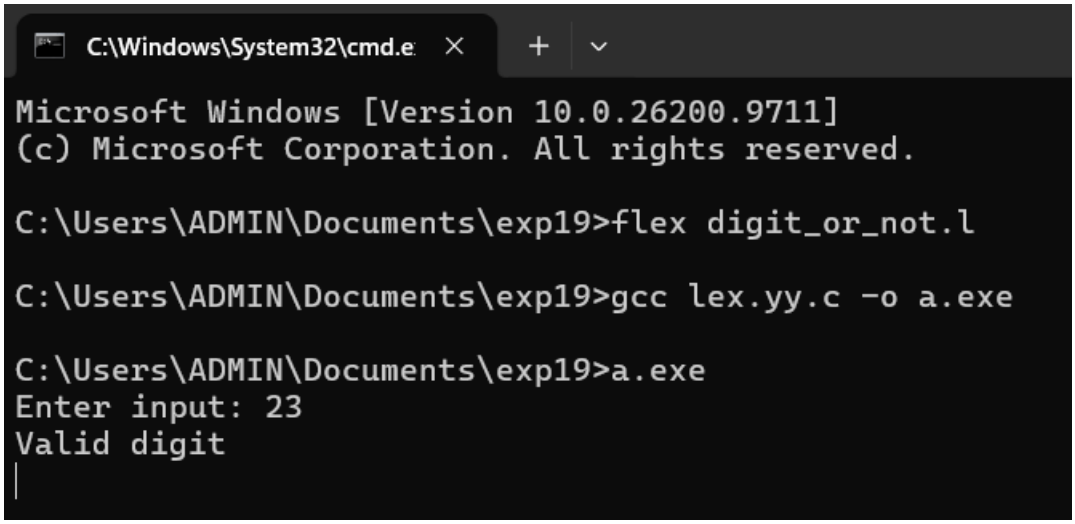
%%
[0-9] {
   printf("Input is a digit\n");
}
. {
   printf("Input is not a digit\n");
}
%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Enter a character: ");
   yylex();
   return 0;
```

}

Output:



Result:

Thus, the LEX program to check whether the input is a digit was executed successfully.

Experiment No. 20

Experiment Name: Implement Basic Mathematical Operations using LEX

Aim:

To write a LEX program to perform basic mathematical operations.

Program: (cal.l)

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

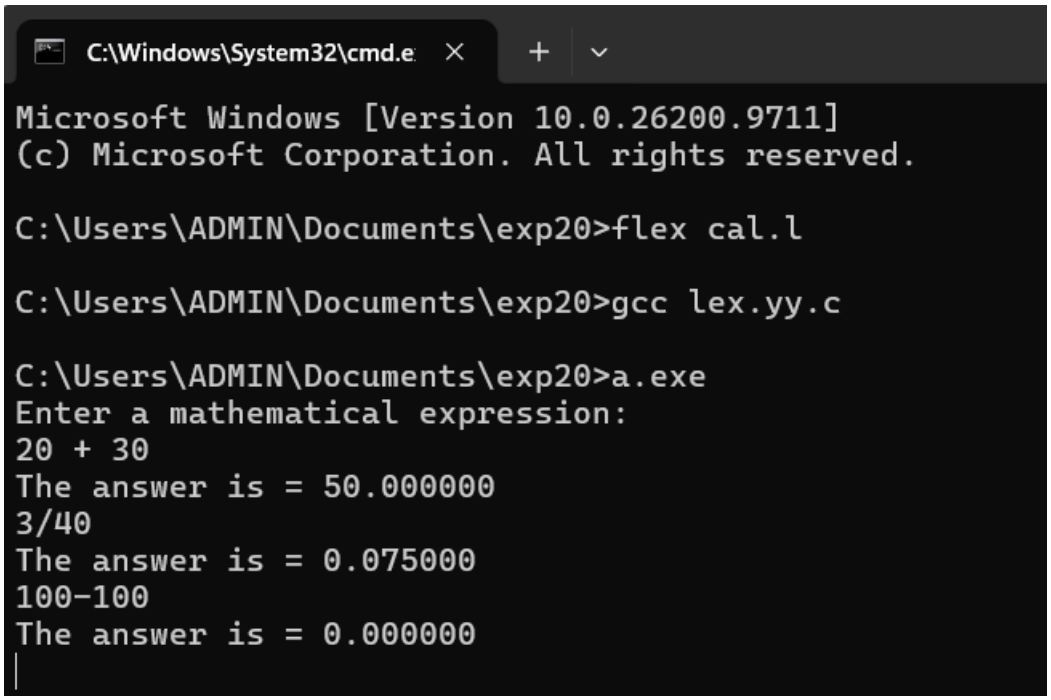
%%
[0-9]+ {
    printf("Number: %s\n", yytext);
}
[+\-*/] {
    printf("Operator: %s\n", yytext);
}
[ \t\n]+ ;
. {
    printf("Invalid character: %s\n", yytext);
}
%%

int yywrap()
{
    return 1;
}

int main()
```

```
{
    printf("Enter a mathematical expression: ");
    yylex();
    return 0;
}
```

Output:



Result:

Thus, the LEX program to recognize basic mathematical operators and numbers was executed successfully.